



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

LAB #3

freeRTOS: TASKS & SYNCHRONIZATION WITH SEMAPHORES

2022 - 2023

freeRTOS: Tasks and Synchronization with Semaphores

- ☐ Overview of RTOS / freeRTOS
- ☐ Task Creation
- ☐ Semaphores
- ☐ Hook functions
- ☐ Interrupts

Overview of RTOS / freeRTOS



www.freertos.org

- ❑ Real time operating system for microcontrollers and small microprocessors.

The screenshot shows the FreeRTOS website homepage. At the top is a navigation bar with links for KERNEL, LIBRARIES, RESOURCES, COMMUNITY, SUPPORT, and a prominent green 'Download FreeRTOS' button. The main heading is 'FreeRTOS™ Real-time operating system for microcontrollers'. Below this is a paragraph describing its development and open-source nature. Two buttons, 'Download FreeRTOS' and 'Getting Started', are provided. A banner for IoT connectivity libraries is visible. The 'Why FreeRTOS?' section highlights three key benefits: a trusted kernel, accelerated time to market, and broad ecosystem support. The 'Features' section at the bottom details the tiny power-saving kernel, support for 40+ architectures, and modular libraries.

FreeRTOS™
Real-time operating system for microcontrollers

Developed in partnership with the world's leading chip companies over a 15-year period, and now downloaded every 175 seconds, FreeRTOS is a market-leading real-time operating system (RTOS) for microcontrollers and small microprocessors. Distributed freely under the MIT open source license, FreeRTOS includes a kernel and a growing set of IoT libraries suitable for use across all industry sectors. FreeRTOS is built with an emphasis on reliability and ease of use.

[Download FreeRTOS](#) [Getting Started](#)

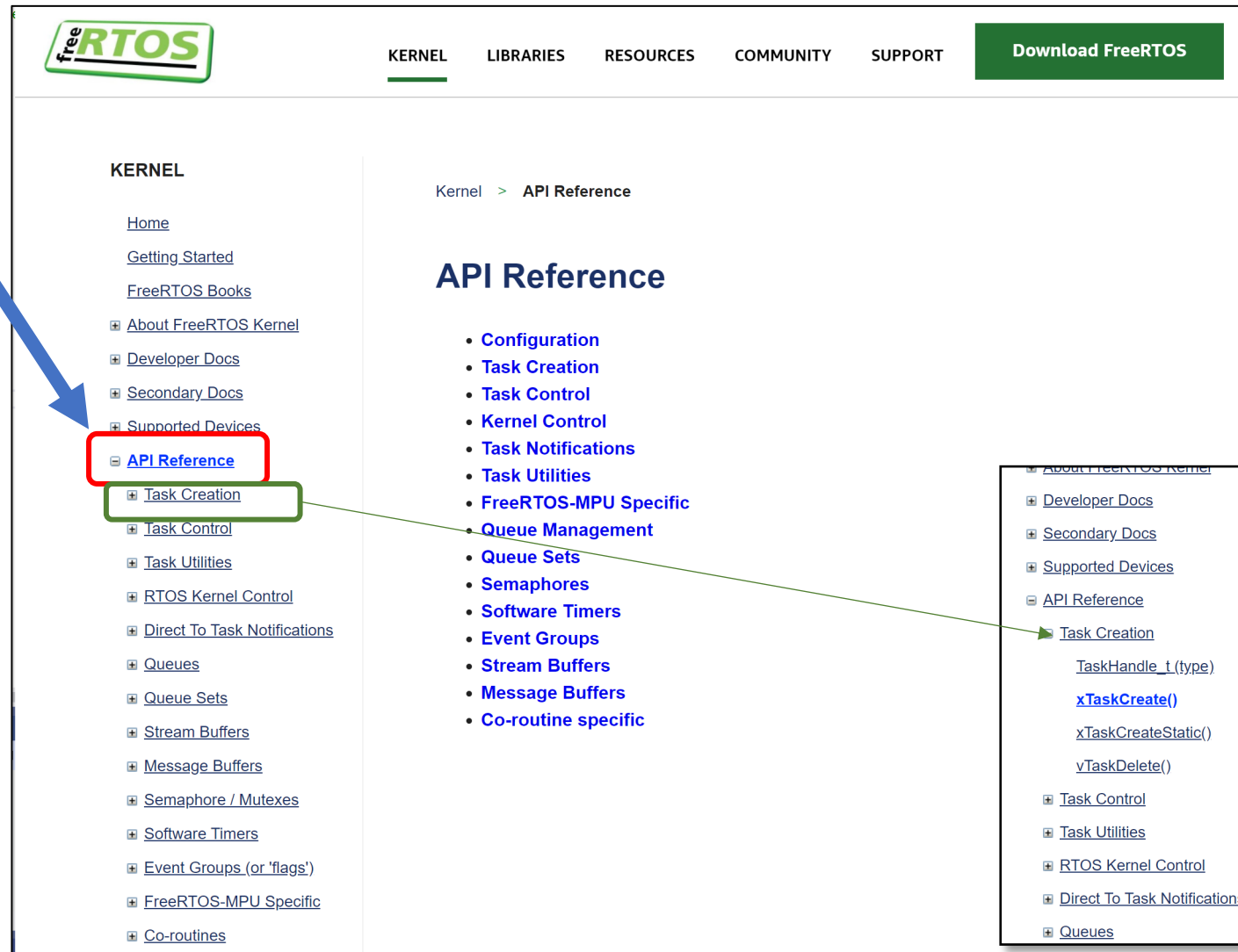
New IoT connectivity & security libraries are now available. [Learn more >](#)

Why FreeRTOS?

- Trusted kernel**
With proven robustness, tiny footprint, and wide device support, the FreeRTOS kernel is trusted by world-leading companies as the de facto standard for microcontrollers and small microprocessors.
- Accelerate time to market**
With detailed pre-configured demos and Internet of Things (IoT) reference integrations, there is no need to determine how to setup a project. Instantly download, compile, and get to market faster.
- Broad ecosystem support**
Our partner ecosystem provides a breadth of options including community contributions, professional support, as well as integrated IDE and productivity tools.

Features

- Tiny, power-saving kernel**
Scalable size, with usable program memory footprint as low as 9KB. Some architectures include a tick-less power saving mode.
- Support for 40+ architectures**
One code base for 40+ MCU architectures and 15+ toolchains, including the latest RISC-V and ARMv8-M (Arm Cortex-M33) microcontrollers.
- Modular libraries**
A growing number of add-on libraries used across all industry sectors, including secure local or cloud connectivity.



freeRTOS

KERNEL LIBRARIES RESOURCES COMMUNITY SUPPORT **Download FreeRTOS**

KERNEL

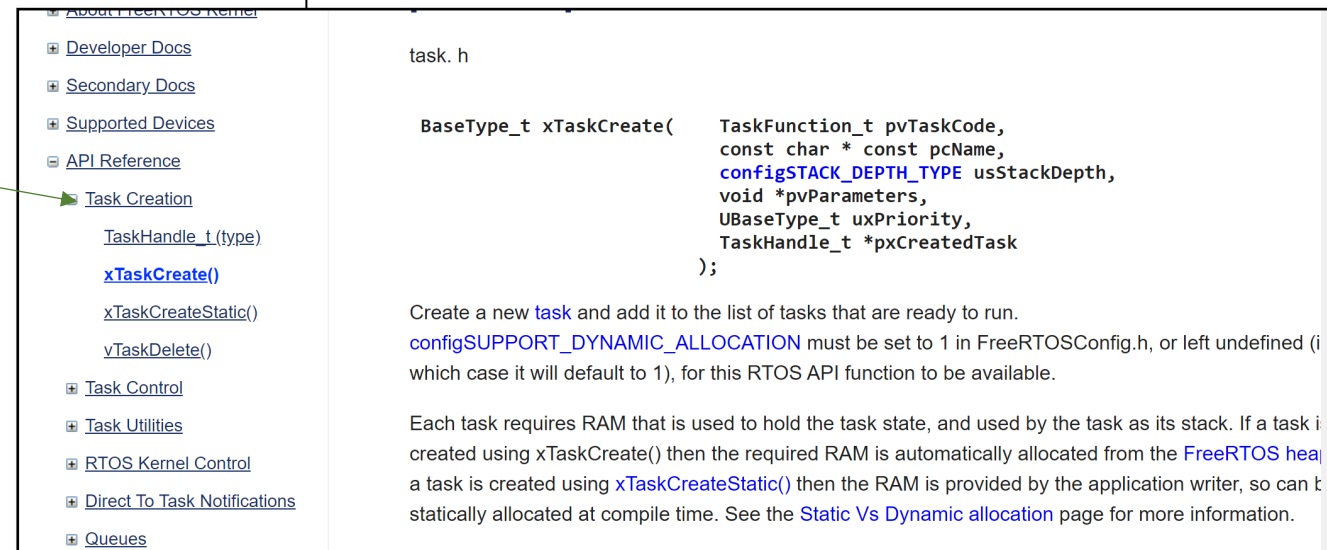
Home
Getting Started
FreeRTOS Books
About FreeRTOS Kernel
Developer Docs
Secondary Docs
Supported Devices
API Reference
Task Creation
Task Control
Task Utilities
RTOS Kernel Control
Direct To Task Notifications
Queues
Queue Sets
Stream Buffers
Message Buffers
Semaphore / Mutexes
Software Timers
Event Groups (or 'flags')
FreeRTOS-MPU Specific
Co-routines

Kernel > **API Reference**

API Reference

- Configuration
- Task Creation
- Task Control
- Kernel Control
- Task Notifications
- Task Utilities
- FreeRTOS-MPU Specific
- Queue Management
- Queue Sets
- Semaphores
- Software Timers
- Event Groups
- Stream Buffers
- Message Buffers
- Co-routine specific

<https://www.freertos.org/a00106.html>



task.h

```

BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,
    const char * const pcName,
    configSTACK_DEPTH_TYPE usStackDepth,
    void *pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pxCreatedTask
);
    
```

Create a new [task](#) and add it to the list of tasks that are ready to run. [configSUPPORT_DYNAMIC_ALLOCATION](#) must be set to 1 in FreeRTOSConfig.h, or left undefined (in which case it will default to 1), for this RTOS API function to be available.

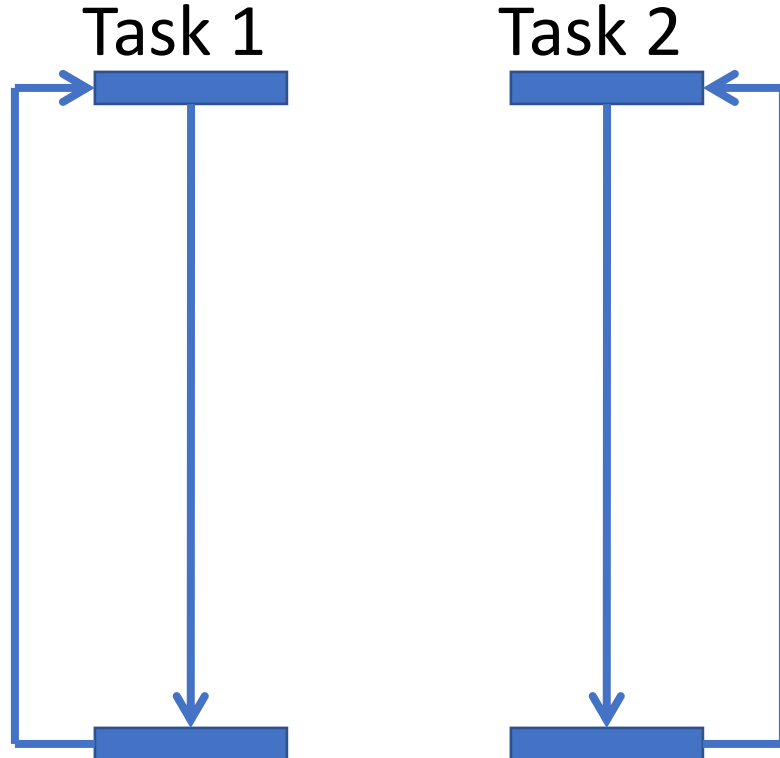
Each task requires RAM that is used to hold the task state, and used by the task as its stack. If a task is created using `xTaskCreate()` then the required RAM is automatically allocated from the [FreeRTOS heap](#). If a task is created using `xTaskCreateStatic()` then the RAM is provided by the application writer, so can be statically allocated at compile time. See the [Static Vs Dynamic allocation](#) page for more information.

- ☐ **Tasks** creation
- ☐ Synchronization with **SEMAPHORES**
- ☐ Message sending/receiving with **MAILBOXES**
- ☐ **Concurrent** (parallel) behavior
- ☐ Can work in a **preemptive** or **cooperative** mode.
- ☐ In a cooperative way, each task should facilitate other tasks to run by invoking **vTaskDelay(xDelay)**, **taskYIELD()** or by using **synchronization mechanisms...**

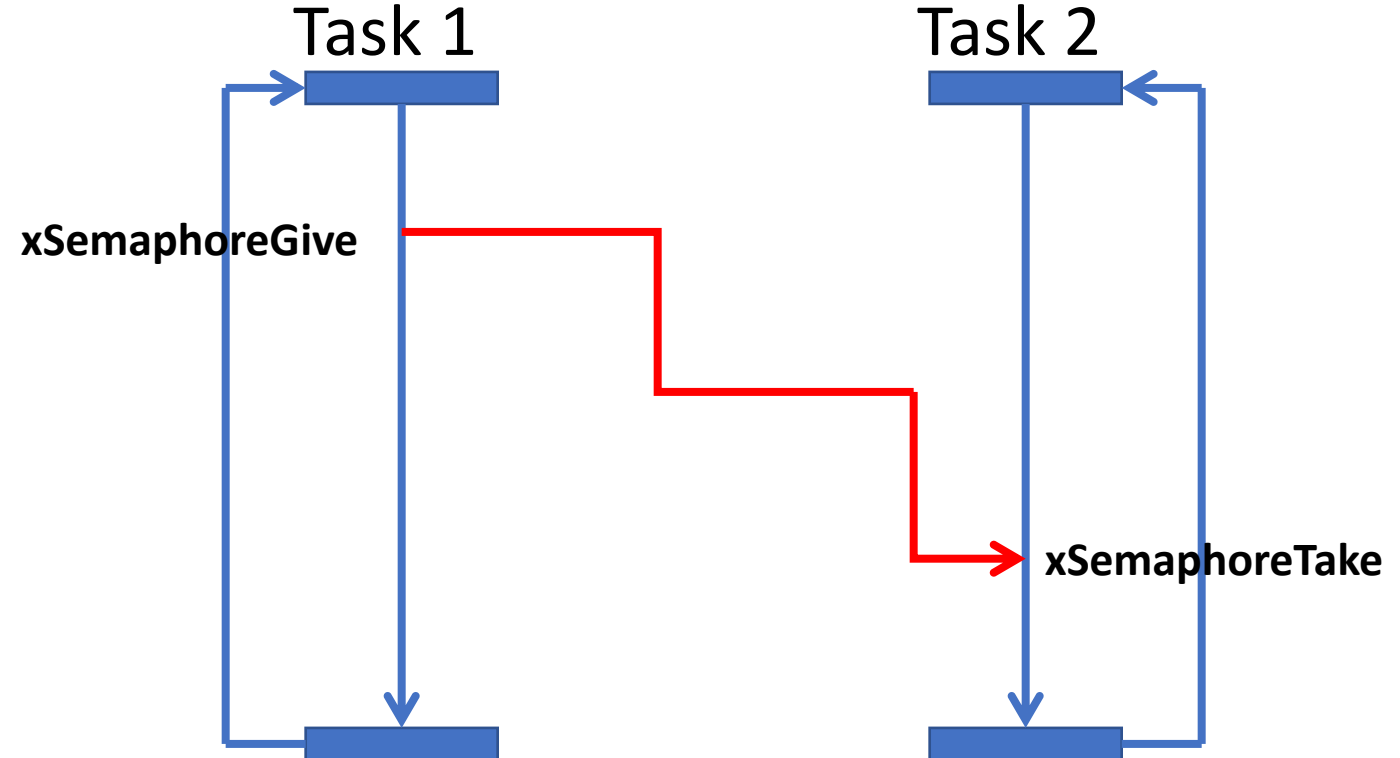
TASK CREATION: Setting up project for running parallel tasks



□ Parallel tasks running **INDEPENDENTLY**



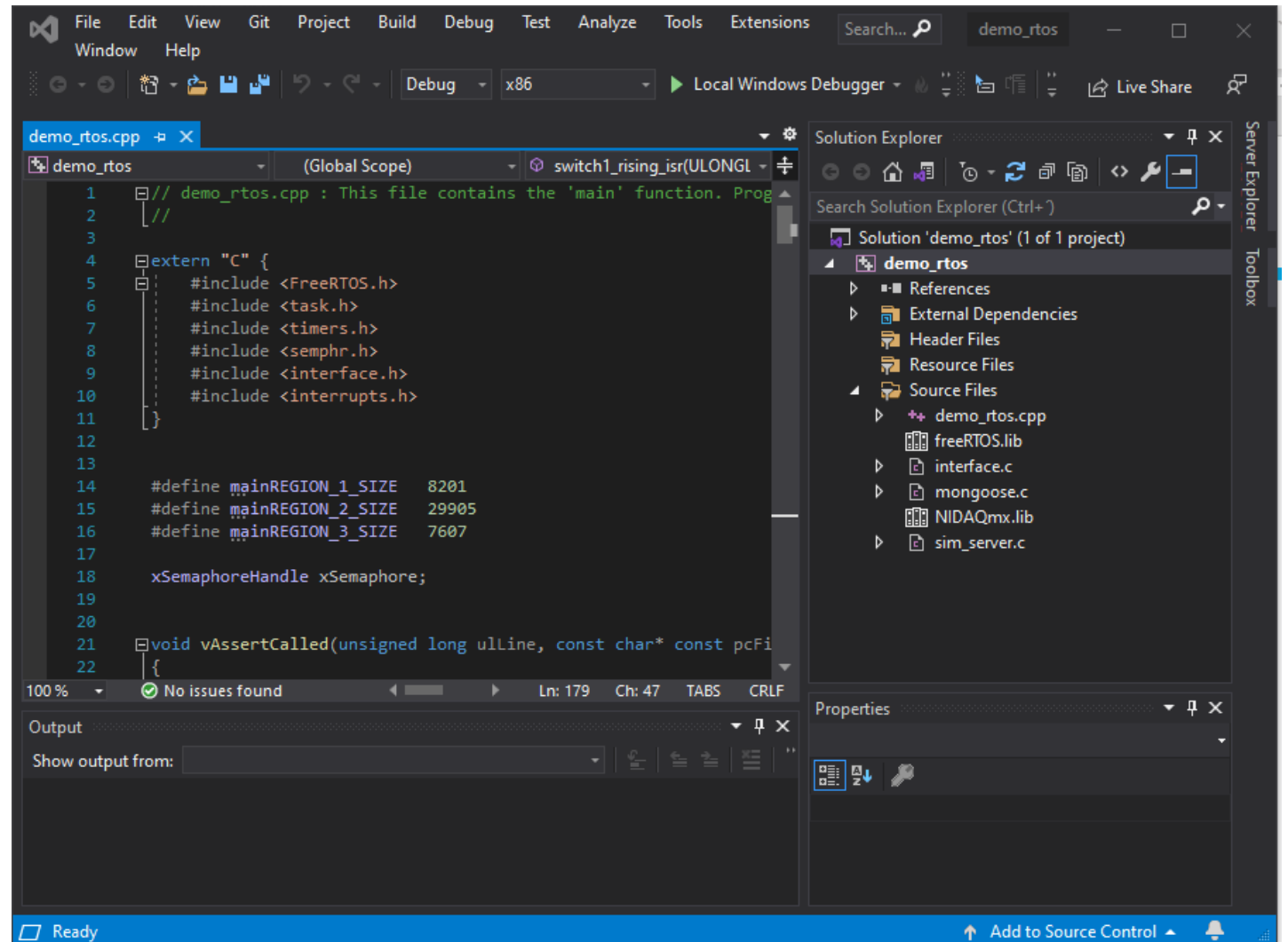
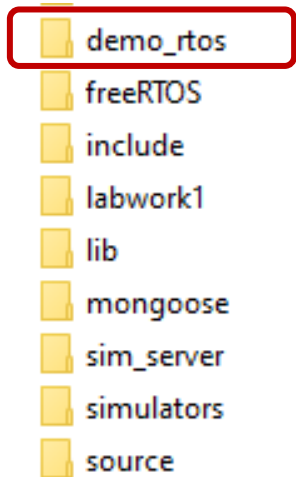
□ Parallel tasks with **SYNCHRONIZATION**



Open demo_rtos project



- Open the **demo_rtos** project from **c:\str** folder:



TASK CREATION: Setting up project for running parallel tasks



❑ Prototypes and macros for heap initialization

```
extern "C" {  
    #include <FreeRTOS.h>  
    #include <task.h>  
    #include <timers.h>  
    #include <semphr.h>  
    #include <interface.h>  
    #include <interrupts.h>  
}  
  
#define mainREGION_1_SIZE 8201  
#define mainREGION_2_SIZE 29905  
#define mainREGION_3_SIZE 7607
```

❑ Heap initialization

```
static void initialiseHeap(void)  
{  
    static uint8_t ucHeap[configTOTAL_HEAP_SIZE];  
    /* Just to prevent 'condition is always true' warnings in configASSERT(). */  
    volatile uint32_t ulAdditionalOffset = 19;  
    const HeapRegion_t xHeapRegions[] =  
    {  
        /* Start address with dummy offsetsSize */  
        { ucHeap + 1, mainREGION_1_SIZE },  
        { ucHeap + 15 + mainREGION_1_SIZE, mainREGION_2_SIZE },  
        { ucHeap + 19 + mainREGION_1_SIZE +  
          mainREGION_2_SIZE, mainREGION_3_SIZE },  
        { NULL, 0 }  
    };  
  
    configASSERT((ulAdditionalOffset +  
        mainREGION_1_SIZE +  
        mainREGION_2_SIZE +  
        mainREGION_3_SIZE) < configTOTAL_HEAP_SIZE);  
    /* Prevent compiler warnings when configASSERT() is not defined. */  
    (void)ulAdditionalOffset;  
    vPortDefineHeapRegions(xHeapRegions);  
}
```

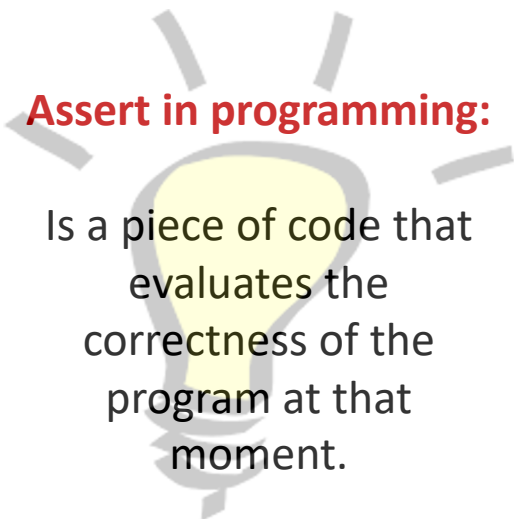
TASK CREATION: Setting up project for running parallel tasks



❑ Assert function for freeRTOS error handling

```
void vAssertCalled( unsigned long ulLine, const char * const pcFileName )
{
    static BaseType_t xPrinted = pdFALSE;
    volatile uint32_t ulSetToNonZeroInDebuggerToContinue = 0;
    /* Called if an assertion passed to configASSERT() fails. See
    http://www.freertos.org/a00110.html#configASSERT for more information. */
    /* Parameters are not used. */
    ( void ) ulLine;
    ( void ) pcFileName;
    printf( "ASSERT! Line %ld, file %s, GetLastError() %ld\r\n", ulLine, pcFileName, GetLastError());

    taskENTER_CRITICAL();
    {
        /* Cause debugger break point if being debugged. */
        __debugbreak();
        /* You can step out of this function to debug the assertion by using
        the debugger to set ulSetToNonZeroInDebuggerToContinue to a non-zero
        value. */
        while( ulSetToNonZeroInDebuggerToContinue == 0 )
        {
            __asm{ NOP };
            __asm{ NOP };
        }
    }
    taskEXIT_CRITICAL();
}
```

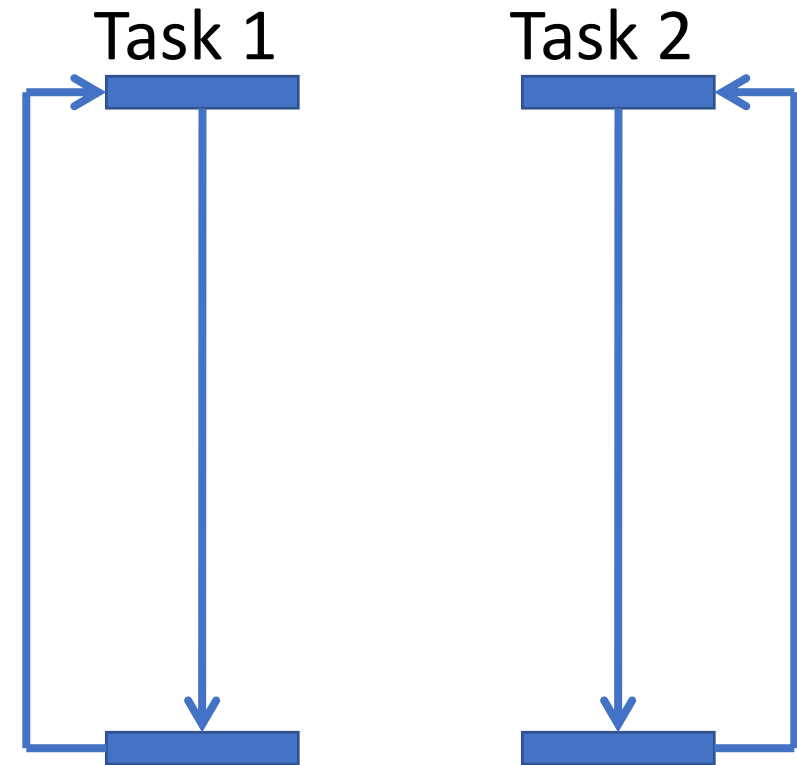


TASK CREATION: Parallel tasks running INDEPENDENTLY



```
void vTaskCode_1( void * pvParameters )
{
    for( ;; )    {
        printf("\nHello from TASK_1");
        // Although the kernel is in preemptive mode,
        // we should help switch to another
        // task with e.g. vTaskDelay(0) or taskYIELD()
        taskYIELD();
    }
}
```

```
void vTaskCode_2( void * pvParameters )
{
    for( ;; )
    {
        printf("\nHello from TASK_2..");
        taskYIELD();
    }
}
```



Nova

```
int main()
{
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vTaskStartScheduler();
}
```



[KERNEL](#)
[LIBRARIES](#)
[RESOURCES](#)
[COMMUNITY](#)
[SUPPORT](#)
[Download FreeRTOS](#)

- [Home](#)
- [Getting Started](#)
- [FreeRTOS Books](#)
- [About FreeRTOS Kernel](#)
- [Developer Docs](#)
- [Secondary Docs](#)
- [Supported Devices](#)
- [API Reference](#)
 - [Task Creation](#)
 - [TaskHandle_t \(type\)](#)
 - [xTaskCreate\(\)](#)
 - [xTaskCreateStatic\(\)](#)
 - [vTaskDelete\(\)](#)
 - [Task Control](#)

xTaskCreate

[Task Creation]

```

BaseType_t xTaskCreate(    TaskFunction_t pvTaskCode,
                          const char * const pcName,
                          configSTACK_DEPTH_TYPE usStackDepth,
                          void *pvParameters,
                          UBaseType_t uxPriority,
                          TaskHandle_t *pxCreatedTask
                          );

```

Create a new [task](#) and add it to the list of tasks that are ready to run.

`configSUPPORT_DYNAMIC_ALLOCATION` must be set to 1 in `FreeRTOSConfig.h`, or left undefined (in which case it will default to 1), for this RTOS API function to be available.

[illegible]

TASK CREATION: Parallel tasks with SYNCHRONIZATION

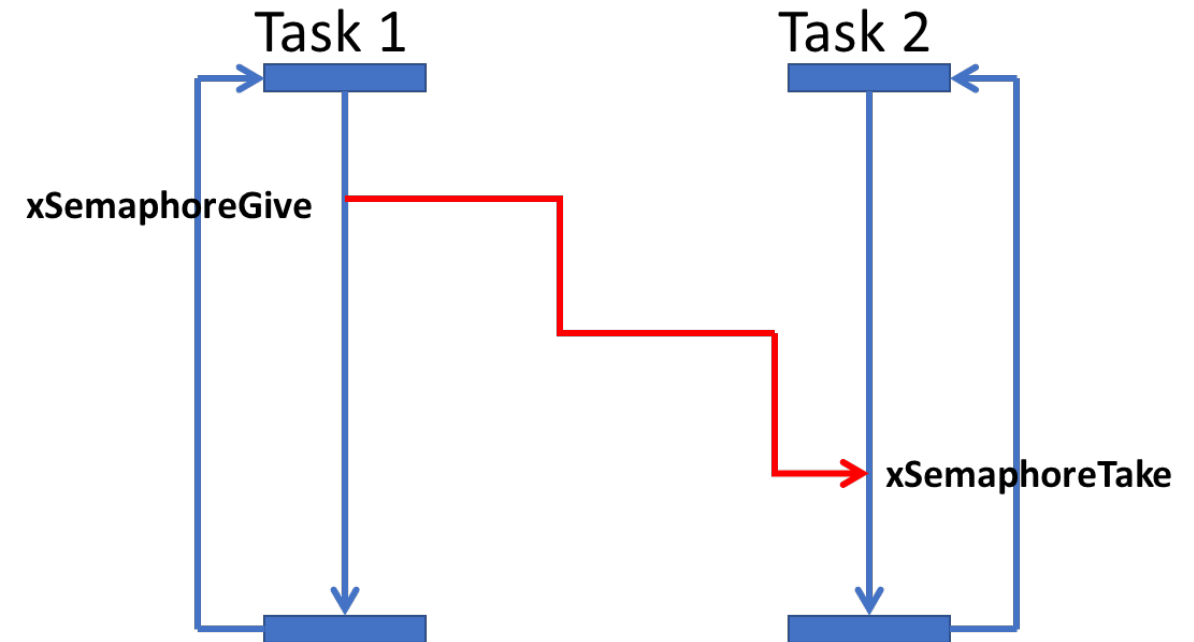


```
// declare the semaphore on top of the file, just below #defines
```

```
xSemaphoreHandle xSemaphore = NULL;
```

```
void vTask_waits(void* pvParameters)
{
    while (1) {
        if (xSemaphoreTake(xSemaphore, 10000) == pdTRUE) //P
            printf("\nvTask_waits has been awoken\n");
        else
            printf("\nvTask_waits is tired of waiting\n");
    }
}
```

```
void vTask_signals(void* pvParameters)
{
    printf("write something\n");
    while (1) {
        char ch[101];
        gets_s(ch, 100); // input from keyboard
        printf("\n vTask_signals got string '%s' from keyboard...", ch);
        if (strcmp(ch, "fim") == 0)
            exit(0); // terminate program...
        xSemaphoreGive(xSemaphore); //V
    }
}
```



TASK CREATION: Parallel tasks with SYNCHRONIZATION



```
void myDaemonTaskStartupHook(void) {  
    // runs only once;  
    // used for code initialization  
  
    /*xTaskCreate(vTaskCode_2, "Task2", 100, NULL, 0, NULL);  
    xTaskCreate(vTaskCode_1, "Task1", 100, NULL, 0, NULL);*/  
  
    xSemaphore = xSemaphoreCreateCounting(10, 0);  
    xTaskCreate(vTask_waits, "vTask_waits", 100, NULL, 0, NULL);  
    xTaskCreate(vTask_signals, "vTaskCode2", 100, NULL, 0, NULL);  
}
```

```
int main()  
{  
    initialiseHeap();  
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;  
    vTaskStartScheduler();  
}
```

```
Microsoft Visual Studio Debug Console  
  
write something  
wake  
  
vTask_signals got string 'wake' from keyboard!  
vTask_waits has been awakened  
vTask_waits is tired of waiting  
hi  
  
vTask_signals got string 'hi' from keyboard!  
vTask_waits has been awakened  
fim  
  
vTask_signals got string 'fim' from keyboard!  
C:\str\example_rtos\Debug\example_rtos.exe (process 23992) exited with
```

```
void vTaskHorizontal(void* pvParameters)
{
    while (TRUE)
    {
        //go right
        uInt8 aa = readDigitalU8(2);
        writeDigitalU8(2, (aa & (0xff - 0x40)) | 0x80);

        // wait till last sensor
        while (readDigitalU8(0) & 0x01) {
            taskYIELD();
        }
        // go left
        aa = readDigitalU8(2);
        vTaskDelay(10); // to simulate some latency
        writeDigitalU8(2, (aa & (0xff - 0x80)) | 0x40);

        // wait till last sensor
        while ((readDigitalU8(0) & 0x04)) {
            taskYIELD();
        }
    }
}
```

Task Creation

```
void vTaskVertical(void* pvParameters)
{
    while (TRUE)
    {
        //go up
        uInt8 aa = readDigitalU8(2);
        writeDigitalU8(2, (aa & (0xff - 0x04)) | 0x08);

        // wait till last sensor
        while ((readDigitalU8(0) & 0x40)) { vTaskDelay(1); }

        // go left
        aa = readDigitalU8(2);
        vTaskDelay(10); // to simulate some latency
        writeDigitalU8(2, (aa & (0xff - 0x08)) | 0x04);

        // wait till last sensor
        while ((readDigitalU8(1) & 0x08)) { vTaskDelay(1); }
    }
}
```

Using freeRTOS with the Kit



```
void inicializarPortos() {  
    printf("\nwaiting for hardware simulator...");  
    printf("\nReminding: gotoXZ requires kit calibration first...");  
    createDigitalInput(0);  
    createDigitalInput(1);  
    createDigitalOutput(2);  
    writeDigitalU8(2, 0);  
    printf("\ngot access to simulator...");  
}
```

```
void myDaemonTaskStartupHook(void) {  
    // runs only once;  
    // used for code initialization  
  
    /*xTaskCreate(vTaskCode_2, "Task2", 100, NULL, 0, NULL);  
    xTaskCreate(vTaskCode_1, "Task1", 100, NULL, 0, NULL);*/  
  
    xSemaphore = xSemaphoreCreateCounting(10, 0);  
    xTaskCreate(vTask_waits, "vTask_waits", 100, NULL, 0, NULL);  
    xTaskCreate(vTask_signals, "vTaskCode2", 100, NULL, 0, NULL);  
  
    inicializarPortos();  
  
    xTaskCreate(vTaskVertical, "vTaskVertical", 100, NULL, 0, NULL);  
    xTaskCreate(vTaskHorizontal, "vTaskHorizontal", 100, NULL, 0, NULL);  
}
```

```
int main()  
{  
    initialiseHeap();  
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;  
    vTaskStartScheduler();  
}
```

Using freeRTOS with the Kit



Running the project...

C:\str\demo_rtos\Debug\demo_rtos.exe

```
waiting for hardware simulator...
Reminding: gotoXZ requires kit calibration first...
Starting web server on port 8081

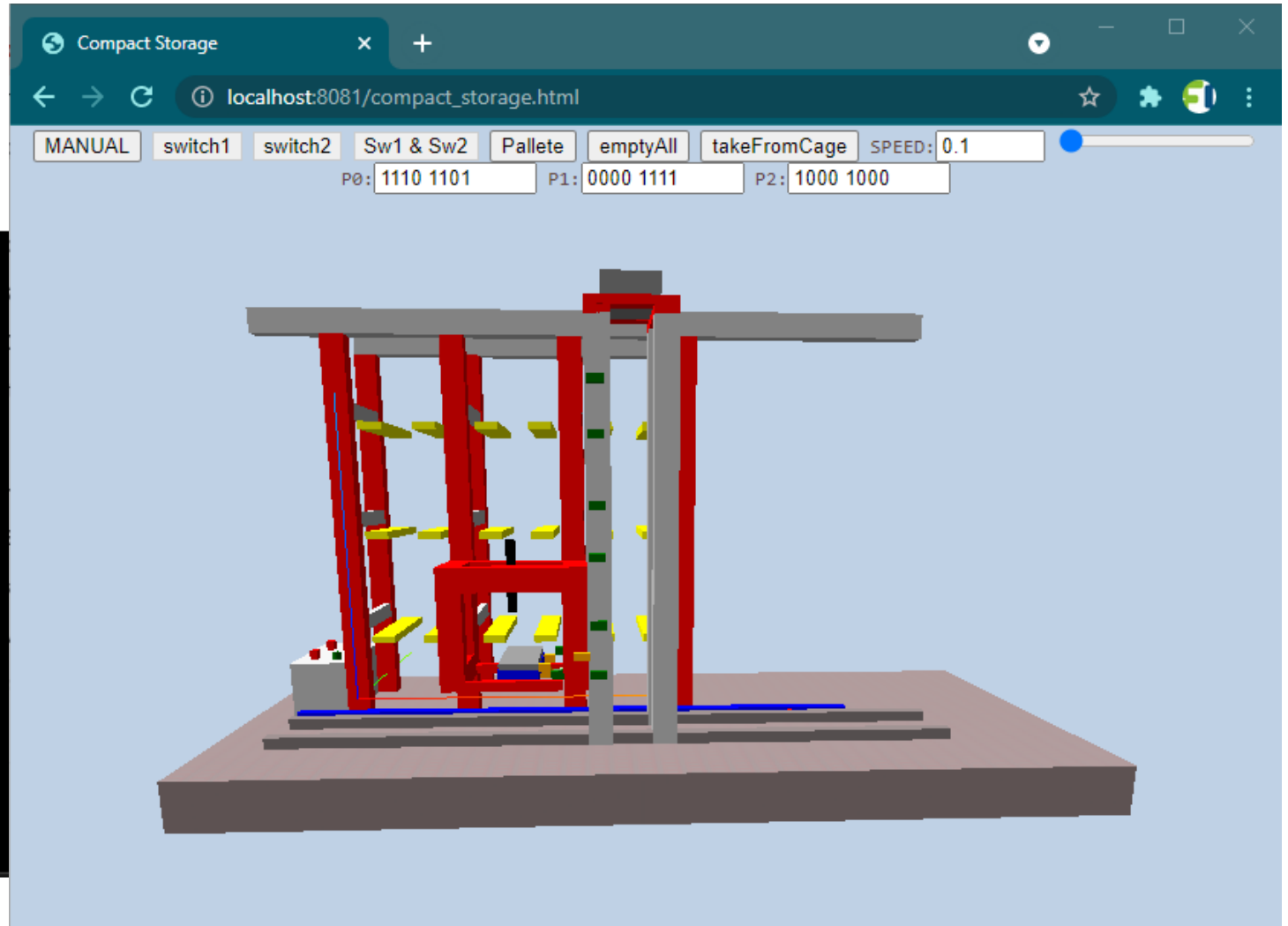
got access to simulator...write something
tommy

vTask_signals got string 'tommy' from keyboardà
vTask_waits has been awaken
and

vTask_signals got string 'and' from keyboardà
vTask_waits has been awaken
jerry

vTask_signals got string 'jerry' from keyboardà
vTask_waits has been awaken

vTask_waits is tired of waiting
```



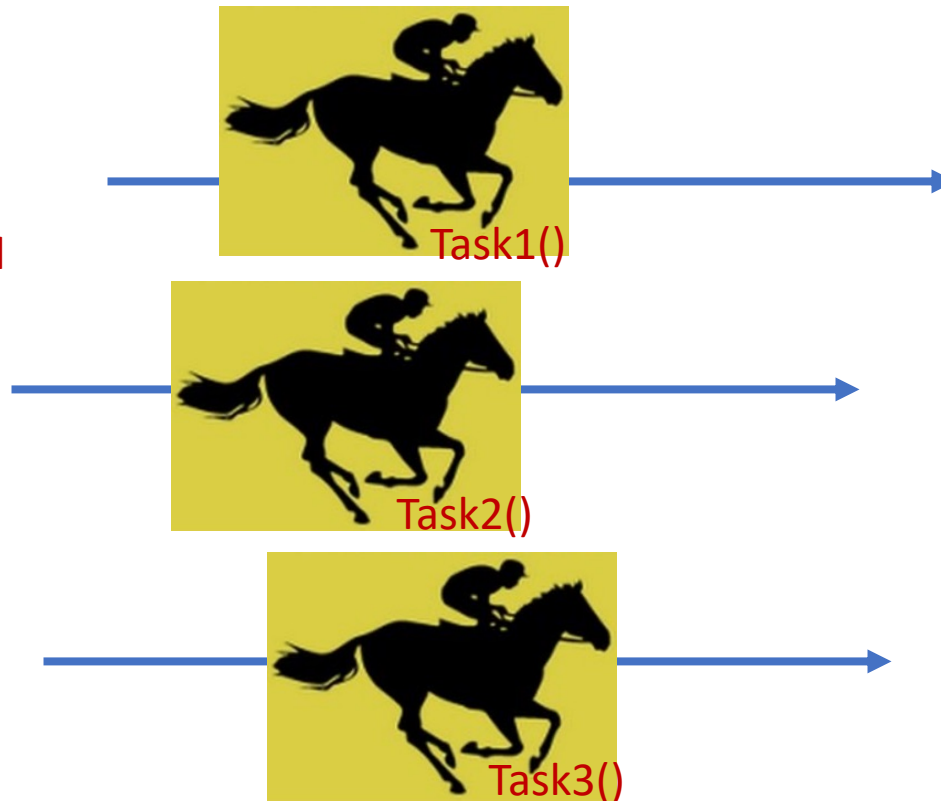
- ❑ Sometimes, an axis stops moving.
- ❑ Erratic/irregular movements may appear!
- ❑ As vertical and horizontal behaviors are running in “parallel”.
- ❑ Both tasks read and write values in port 2 “simultaneously”.
- ❑ And so, the values sent to port 2 are overlapping on each one!



Race Condition

- Several tasks running in parallel and doing simultaneous things, which may interfere with each other

Either correct or
potential undesired
behavior



A **race condition** or **race hazard** is the condition of an [electronics](#), [software](#), or other [system](#) where the system's substantive behavior is dependent on the sequence or timing of other uncontrollable events. It becomes a [bug](#) when one or more of the possible behaviors is undesirable.

The term **race condition** was already in use by 1954, for example in [David A. Huffman](#)'s doctoral thesis "The synthesis of sequential switching".

https://en.wikipedia.org/wiki/Race_condition

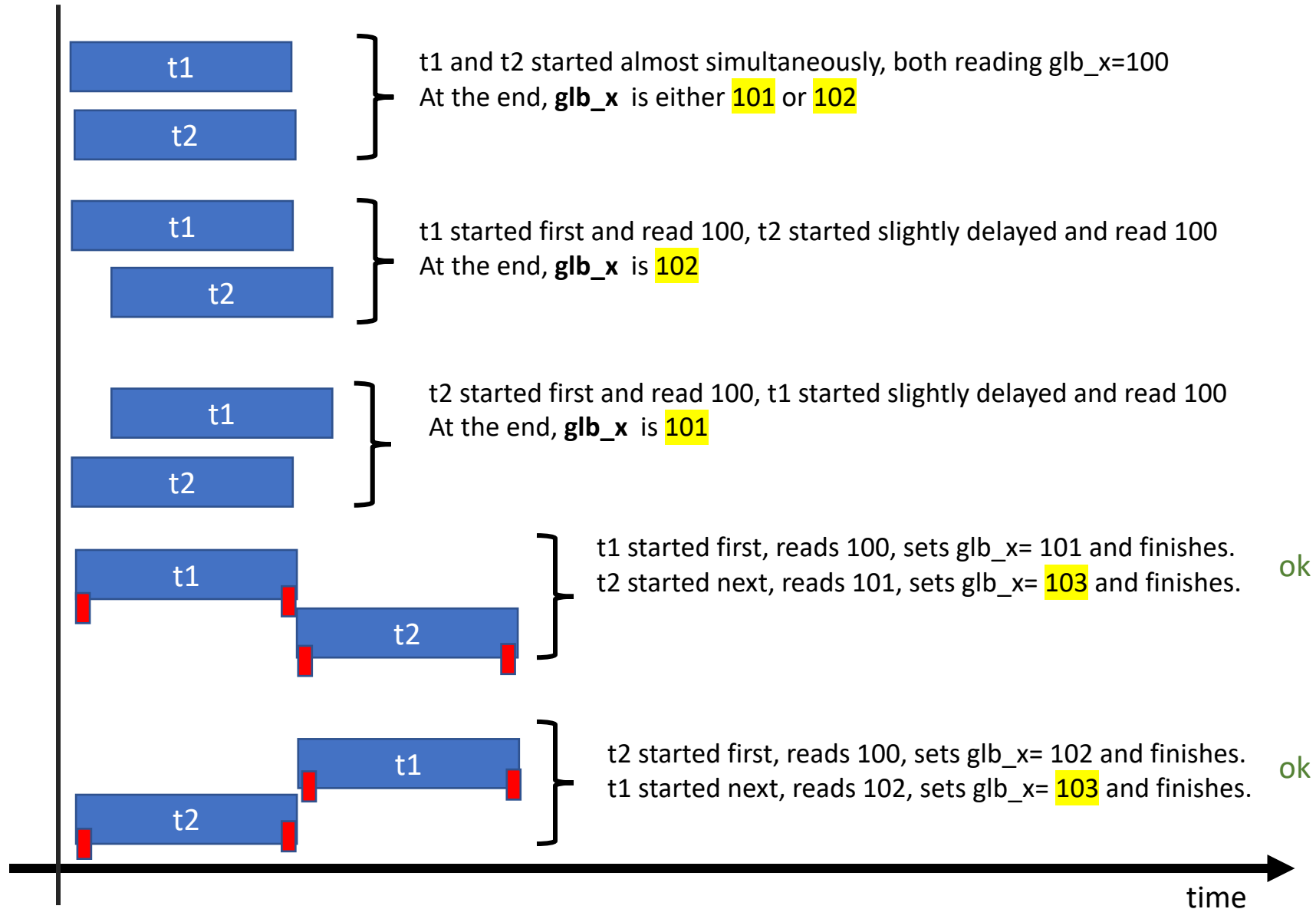
- Let's suppose `glbl_x=100` and two tasks (t1 and t2) running “almost exactly” at the same time.

```
void t1() {  
    int aux= glbl_x;  
    aux=aux+1;  
    glbl_x = aux;  
}
```

```
void t2() {  
    int aux= glbl_x;  
    aux=aux+2;  
    glbl_x = aux;  
}
```

- After execution of t1 and t2, the global `glb_x` resource may assume the following values: 101, 102, or 103.

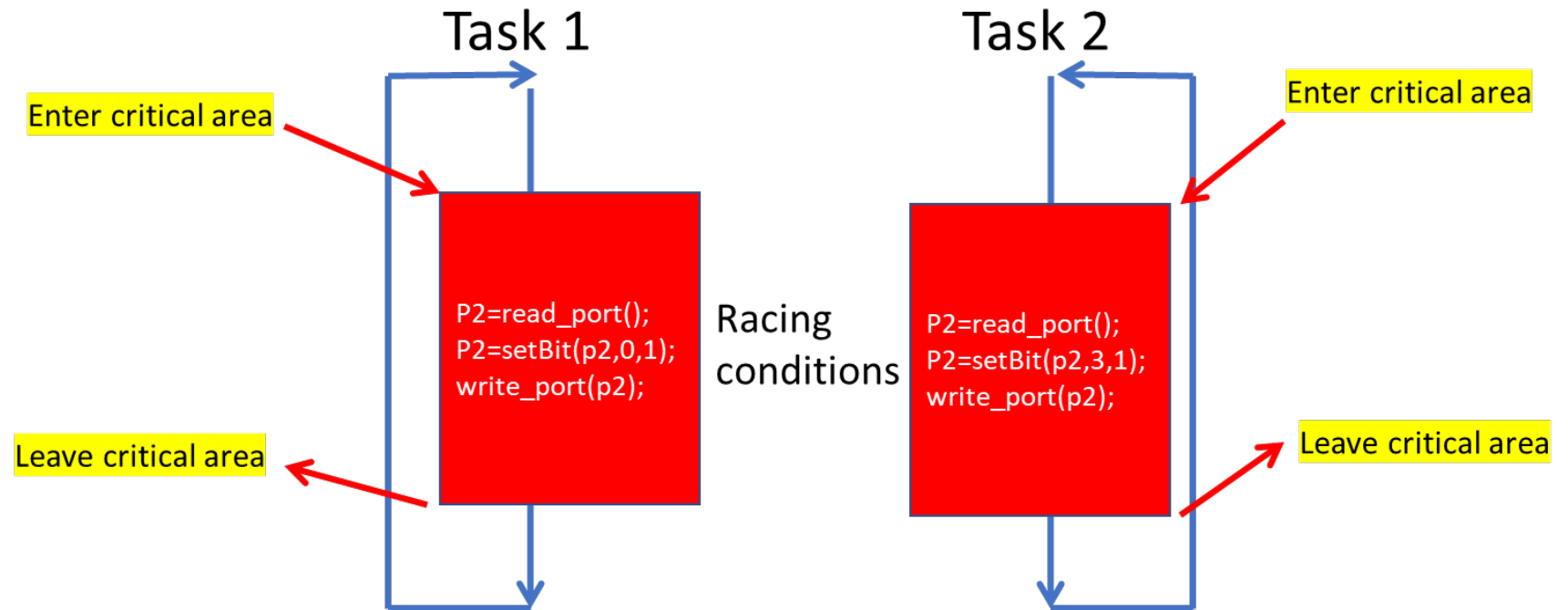
Race Condition



■ Synchronization to ensure that t2 starts after the t1 finishes or vice-versa

Race Condition: Storage Kit (access to Port 2)

- ❑ Task 1 and Task 2 are running a fragment of code, which cannot be executed simultaneously
- ❑ We need to specify **critical areas / sections**.
- ❑ A **critical area** is a block of code which while running, everything else in the system freezes until the code inside the block finishes and execution exits from that area.



Race Condition: Storage Kit (access to Port 2)

- ❑ `taskENTER_CRITICAL()` and `taskEXIT_CRITICAL()` operate at kernel level, meaning when inside a critical area, “everything” else inside the kernel stops working until exiting the critical area.
- ❑ As such, the code inside critical areas should work as fast as possible (no loops, no sleeps,... no anything causing latency).
- ❑ For less strident conditions, the recommended alternative is to model mutual exclusion with semaphores.

```
void vTaskVertical(void* pvParameters)
{
    while (TRUE)
    {
        //go up
        taskENTER_CRITICAL();
        uint8 aa = readDigitalU8(2);
        writeDigitalU8(2, (aa & (0xff - 0x04)) | 0x08);
        taskEXIT_CRITICAL();

        // wait till last sensor
        while ((readDigitalU8(0) & 0x40)) { taskYIELD(); }

        // go left
        taskENTER_CRITICAL();
        aa = readDigitalU8(2);
        vTaskDelay(10); // simulate some latency
        writeDigitalU8(2, (aa & (0xff - 0x08)) | 0x04);
        taskEXIT_CRITICAL();

        // wait till last sensor
        while ((readDigitalU8(1) & 0x08)) { taskYIELD(); }
    }
}
```

❑ Using `vApplicationIdleHook`

- It runs when all tasks are occupied (waiting for something)

```
void myIdleHook(void) {  
    // do something here  
    printf("\nidle hook...");  
    Sleep(0);  
}
```

```
int main()  
{  
    initialiseHeap();  
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;  
    vApplicationIdleHook = &myIdleHook;  
    vTaskStartScheduler();  
}
```

❑ Using `vApplicationTickHook`

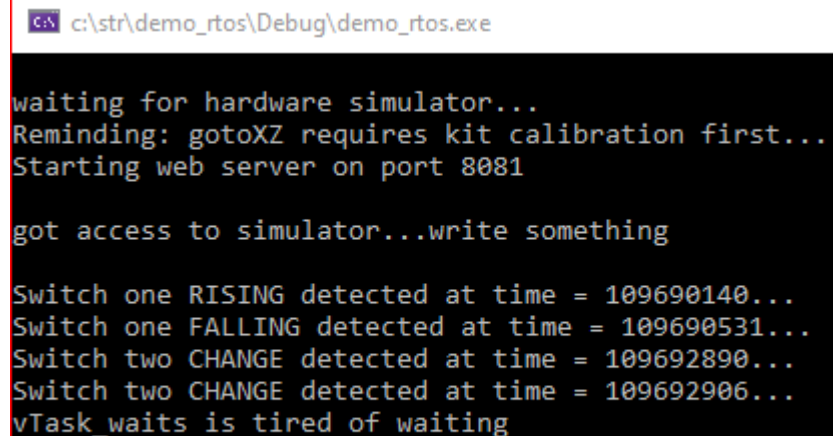
- It is useful to performing periodic operations

```
void myTickHook(void) {  
    // do something here  
    printf("\ntick hook...");  
    Sleep(4000);  
}
```

```
int main()  
{  
    initialiseHeap();  
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;  
    vApplicationTickHook = &myTickHook;  
    vApplicationIdleHook = &myIdleHook;  
    vTaskStartScheduler();  
}
```

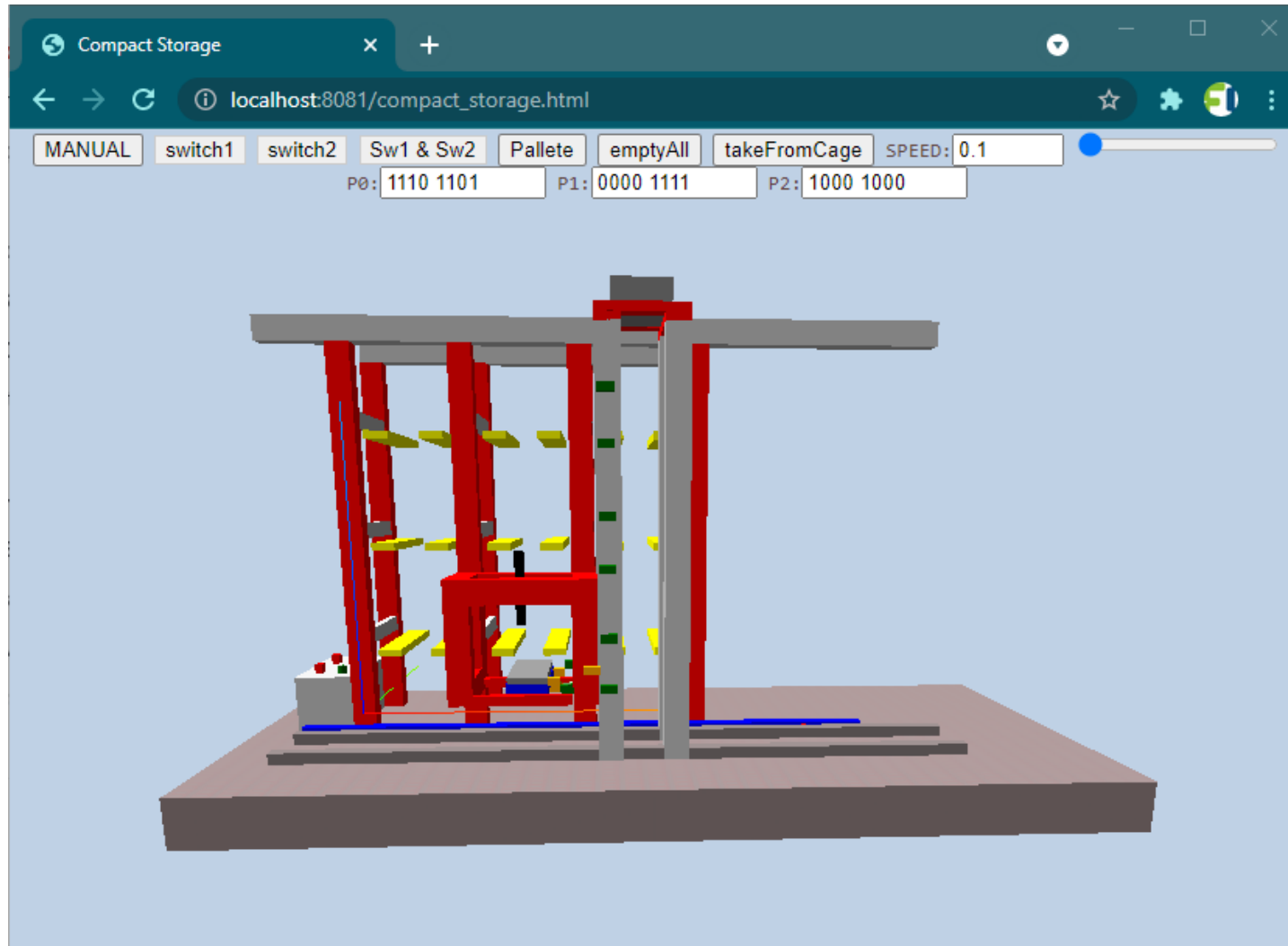
❑ Using Interrupt Service Routines (ISR) (on port bit values)

```
void switch1_rising_isr(ULONGLONG lastTime) {  
    // GetTickCount64() current time in milliseconds  
    // since the system has started...  
    ULONGLONG time = GetTickCount64();  
    printf("\nSwitch one RISING detected at time = %llu...", time);  
}  
  
void switch1_falling_isr(ULONGLONG lastTime) {  
    ULONGLONG time = GetTickCount64();  
    printf("\nSwitch one FALLING detected at time = %llu...", time);  
}  
  
void switch2_change_isr(ULONGLONG lastTime) {  
    ULONGLONG time = GetTickCount64();  
    printf("\nSwitch two CHANGE detected at time = %llu...", time);  
}
```



```
c:\str\demo_rtos\Debug\demo_rtos.exe  
  
waiting for hardware simulator...  
Reminding: gotoXZ requires kit calibration first...  
Starting web server on port 8081  
  
got access to simulator...write something  
  
Switch one RISING detected at time = 109690140...  
Switch one FALLING detected at time = 109690531...  
Switch two CHANGE detected at time = 109692890...  
Switch two CHANGE detected at time = 109692906...  
vTask_waits is tired of waiting  
-
```

```
void myDaemonTaskStartupHook(void) {  
    // ...code already here  
    attachInterrupt(1, 5, switch1_rising_isr, RISING);  
    attachInterrupt(1, 5, switch1_falling_isr, FALLING);  
    attachInterrupt(1, 6, switch2_change_isr, CHANGE);  
}
```



- ☐ Start thinking of which tasks are needed to be executed synchronously or in parallel.
- ☐ Tip: start by implementing **task_gotoX** and **task_gotoZ...**
- ☐ Draw a first diagram illustrating the tasks and the interactions between them.



KEEP UP THE
GOOD
WORK!